Question 1 Part (a)

Binary Search, AVL and Splay Trees. (Shaffer Ch. 8 & 13.)

[Shaffer 5.21] Write a recursive function named **checkBST** that, given the pointer to the root of a binary tree, will return `true` if the tree is a BST, and `false` if it is not.

```java
/** Check whether a BT (that puts dupes on the right) is a BST */
public boolean checkBST(Node root) {
    return root == null ||
            (allLessThan(root.left, root.value) &&
            allGreaterThan(root.right, root.value) &&
            checkBST(root.left) &&
            checkBST(root.right));
}

private boolean allLessThan(Node node, int value) {
    return node == null ||
      (node.value < value && allLessThan(node.left, value) &&
        allLessThan(node.right, value));
}

/** Allow duplicates on right (per Piazza comment from Dr. McAnn) */
private boolean allGreaterThan(Node node, int value) {
    return node == null ||
      (node.value >= value && allGreaterThan(node.left, value) &&
        allGreaterThan(node.right, value));
}
```

Question 1 Part (b)

> Write a boolean Java method `isAVL()` that, given a reference to the root node of a binary tree, returns true if the tree is a legal AVL tree, or false if it is not. You may call your answer to 1(a) (a.k.a. 5.21), and you may write additional helper methods if you wish.

```java
/** Check whether a BT (that puts dupes on the right) is an AVL */
static boolean isAVL(Node root) {
  return checkBST(root) && isBalanced(root);
}

private static boolean isBalanced(Node node) {
  if (node == null)
    return true;

  int leftHeight = height(node.left);
  int rightHeight = height(node.right);

  return Math.abs(leftHeight - rightHeight) <= 1 &&
      isBalanced(node.left) &&
      isBalanced(node.right);
}

private static int height(Node node) {
  if (node == null)
    return 0;

  return 1 + Math.max(height(node.left), height(node.right));
}
```

Question 1 Part (c)

Build an AVL tree by inserting the following letters, in the order given, into the tree, and show the resulting tree after every third insertion (that is, show the tree after inserting Y, after inserting U, and after inserting N): *K, T, Y, V, W, U, G, E, N*

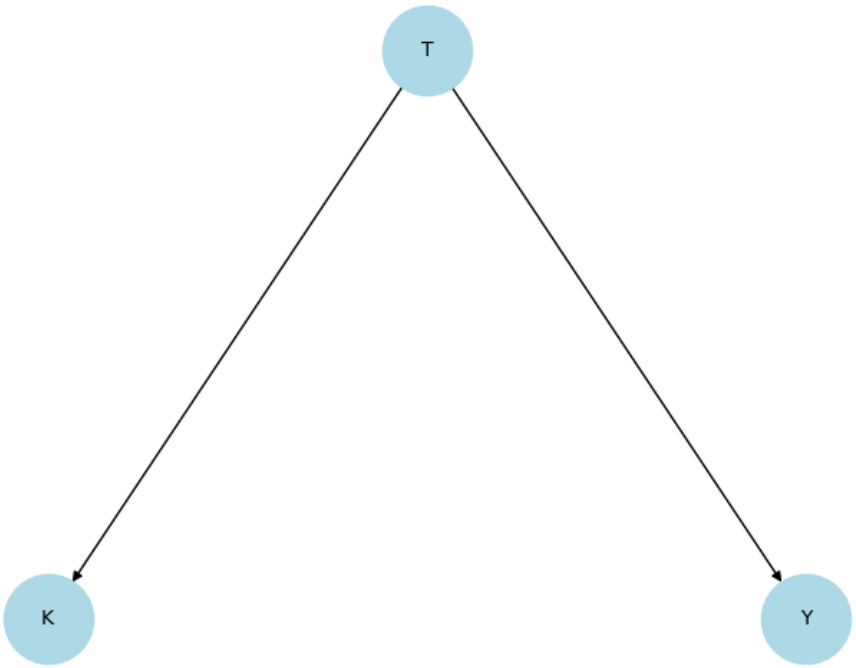


Figure 1: AVL After Step 3 (Insert *Y*)

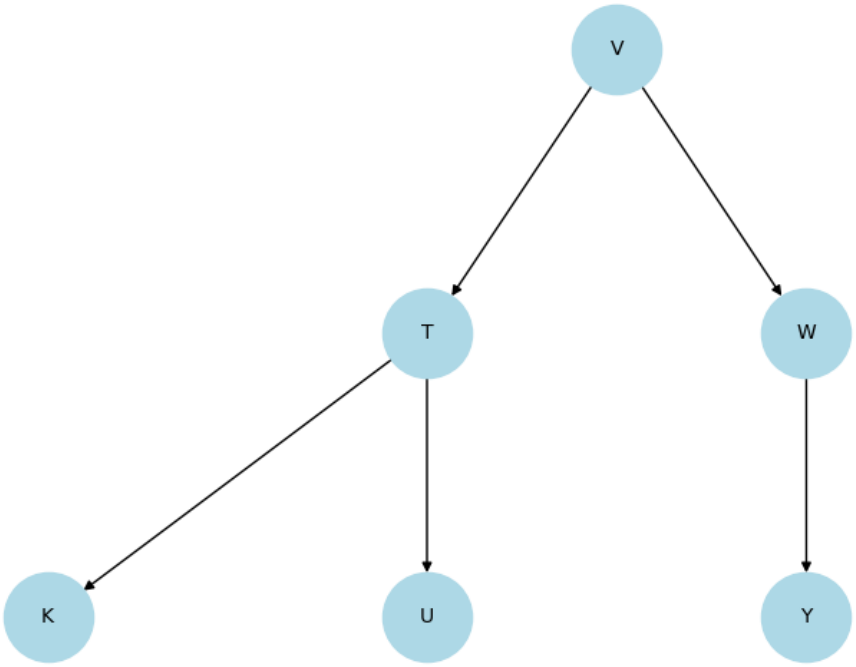


Figure 2: AVL After Step 6 (Insert *U*)

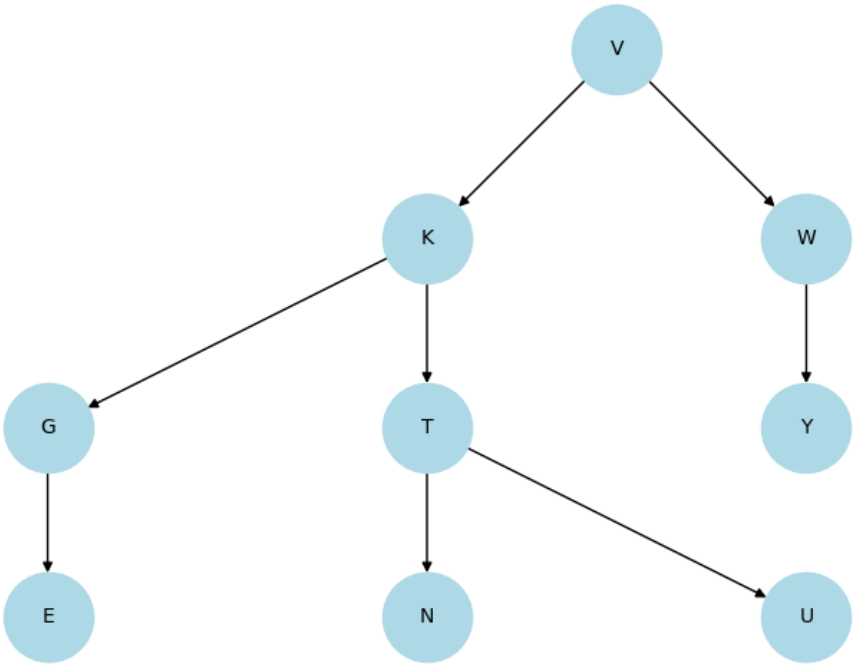


Figure 3: AVL After Step 9 (Insert *N*)

Question 1 Part (d)

Now assume that your final tree from part (c) is a splay tree. Search for *U*, and show the resulting tree after splaying

---

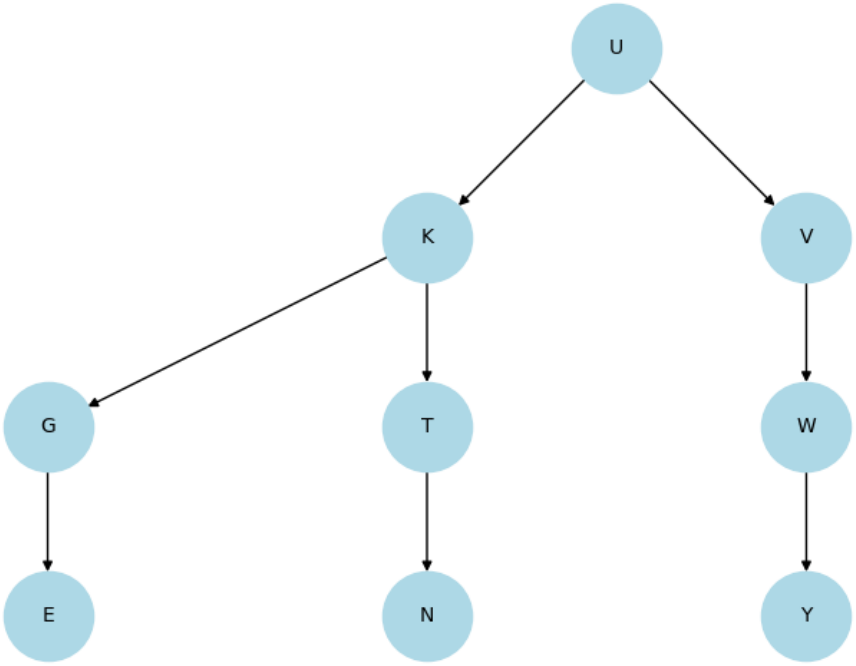


Figure 1: Splay Tree after Searching for *U*

Question 1 Part (e)

Using the tree from the end of part (d), search for *E* in that tree, and show the resulting tree after splaying.

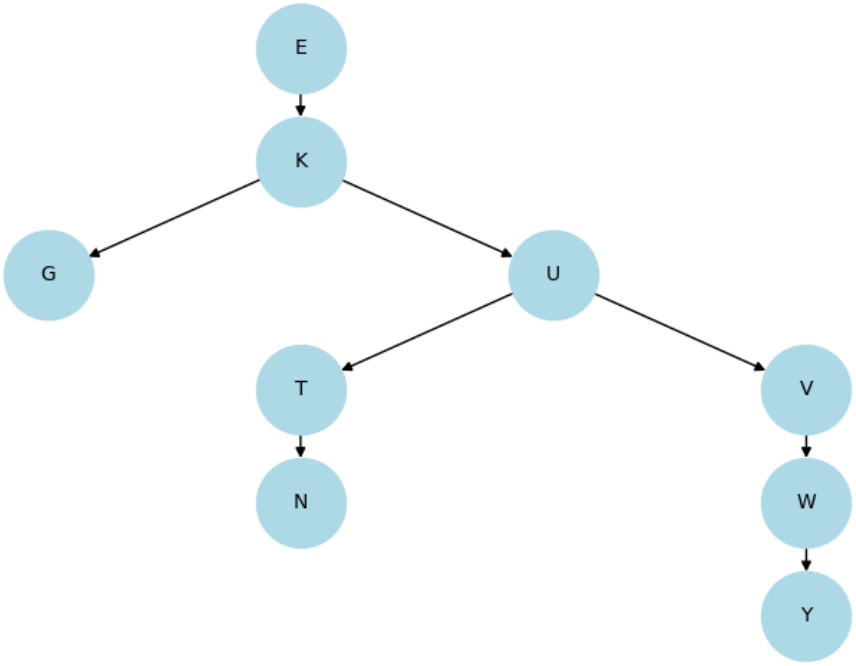


Figure 1: Splay Tree after Searching for *E*

Question 2 Part (a)

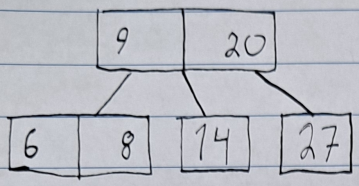
2–3–... Trees. (Shaffer Ch. 10.)

Construct a **2–3** tree from the sequence of data values $6, 9, 20, 8, 14, 27, 43, 22, 30,$ and 19. Show the resulting tree, *as well as* your intermediate tree after the value 27 has been inserted

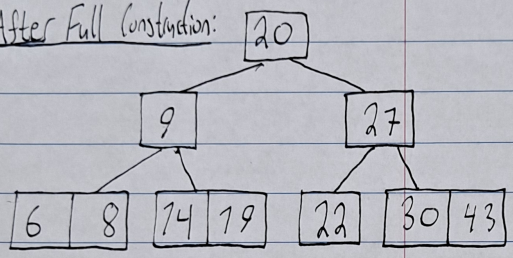
---

Question 2 Part (a)

After 27 Inserted:



After Full Construction:

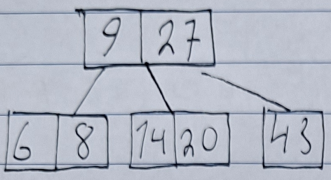


Question 2 Part (b)

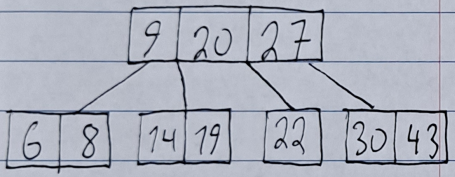
Construct a **2–3–4** tree from the sequence of data values used in part (a). When you need to promote, promote the second–largest value of the over–full node. Show the resulting tree, as well as your intermediate tree after the value 43 has been inserted.

Question 2 Part (b)

After Inserting 43:



After Full Construction:

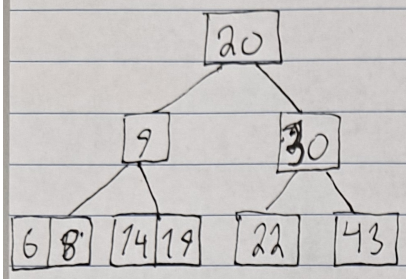


Question 2 Part (c)

From the final **2–3–4** tree from part (b), delete 27 and show the resulting tree

---

Question 2 Part (c)



After deleting 27, assuming no minimum threshold capacity

Question 2 Part (d)

From the tree at the end of part (c), delete 22 and show the resulting tree.

Question 2 Part (d)

After deleting 22: